From Equations to Autonomy: A Pedagogical Roadmap for Designing Formally Verified, Causally Grounded Autonomous Rocket Landing Controllers

Harini S

SCOPE
Vellore Institute of Technology, Chennai
harini.s2023d@vitstudent.ac.in

Gowshick S

SCOPE
Vellore Institute of Technology, Chennai
gowshick.s2023@vitstudent.ac.in

Dr. Kiran Kumar M

SENSE
Vellore Institute of Technology, Chennai

Dr. Aarthi B

SCOPE
Vellore Institute of Technology, Chennai

Abstract

Bringing a reusable launch vehicle to a precise and propulsive touchdown is arguably one of the most technically challenging control problems today in the field of rocketry. While demonstrating their feasibility is one important thing, understanding why a given solution works with the aid of theoretical tools from deep reinforcement learning, causal inference, and formal verification is a much deeper challenge. These tools are currently scattered across disparate fields that rarely intersect, especially in the context of this problem. This paper follows a deliberate pedagogical journey through classical guidance and control theory and leads into the realm of modern learning-based autonomy. We divide the landing problem into its constituent mathematical sub-challenges: six-degree-of-freedom nonlinear dynamics, sparse reward function design, Sim-to-Real transfer, closed-loop stability certification, and runtime safety specification. For each sub-challenge, we determine the appropriate theoretical tool, build intuition around it, and connect it to the existing research. These tools are then assembled to create the *Causal Verifiable Landing* (CVL) conceptual framework, which combines Structural Causal MDPs, Proximal Policy Optimization, GR(1) Temporal Logic Synthesis, Neural Lyapunov, Bayesian World Models, and Online Meta-Learning into a unified architectural blueprint. The paper does not present any experimental results, but its contribution helps students understand why any particular element is included, what it ensures, and how all of such guarantees come together, making it a self-contained entry point to the design of safety-critical autonomous aerospace systems.

Keywords: autonomous landing; deep reinforcement learning; causal inference; formal verification; Lyapunov stability; temporal logic; proximal policy optimization; reusable launch vehicles; Sim-to-Real transfer; pedagogical roadmap

Contents

1. Introduction

The advent of commercial spaceflight has made RLV landing a reality instead of just a theory with specific performance standards. Modern commercial space flight has demonstrated its ability to achieve autonomous retro-propulsive landing with incredible accuracy and repeatability. However, the machinery and reasoning behind this achievement are still proprietary, and the relevant academic literature spanning dynamics modeling, learning control, and safety certification is still fragmented and across communities that do not normally interact with each other.

This fragmentation creates a significant challenge for students interested in this field. Traditional aerospace engineering curricula include classical theory and practice in guidance, navigation, and control (GNC) theory and practice, including convex trajectory optimization, Lossless Convexification (LCvx), and various shaping techniques, but not DRL. Computer science and machine learning programmes go deep on DRL, causal inference, and meta-learning, but virtually never apply these tools to physically realistic, safety-critical dynamics. Formal methods courses introduce temporal logic and model checking in the abstract, with almost no connection to continuous nonlinear control. The student who wants to work at the junction of all three fields — arguably the most consequential junction in modern autonomy research — therefore has no single coherent thread to follow.

This paper aims to bridge this gap. We offer a structured pedagogical decomposition of the rocket landing problem anchored to seven fundamental failure modes of naïve approaches, each paired with the theoretical tool that addresses it. These tools are then synthesised into the Causal Verifiable Landing (CVL) architecture, a carefully reasoned eleven-layer blueprint whose integration logic is explained step by step. CVL is not presented as a validated system, but as a design framework, with every component individually grounded in peer-reviewed literature, and with the connections between components articulated so that a student can follow, challenge, and build on them.

Contributions

The paper will make the following specific contributions:

1. **Problem decomposition:** relating seven distinct failure modes of naïve DRL-based landing controllers to their root causes and suitable theoretical remedies.
2. A **mathematical foundations guide** for 6-DOF dynamics, MDP formulation, causal augmentation, temporal logic safety specification, and Lyapunov stability, all explained at a level suitable for advanced undergraduate students.
3. A **unified architectural blueprint** (CVL) for relating eleven functional layers, including discussion of purpose, inputs, outputs, and failure modes solved per layer.
4. A **literature navigation guide** pointing out the primary papers a student will need to read for each component, ordered by prerequisite depth.
5. A **practical implementation roadmap** showing how a student team can build simplified versions of CVL using open-source tools, starting from a 2-D point-mass simulation and progressing towards the 6-DOF problem.

2. Seven Problems That a Naive Approach Can't Solve

The value of a well-designed solution is best appreciated by first understanding the specific ways in which the naive solutions fail. This section catalogues seven specific difficulties that plague naive DRL-based landing controllers. Each is described clearly because, in the CVL framework, the rationale behind each architectural choice ultimately maps back to one of these failure modes. The process of first specifying the problem is, in fact, part of the larger pedagogical strategy that pervades the remainder of the paper.

2.1. Challenge 1 — Nonlinear Variable-Mass Dynamics

The rocket is not actually a rigid body with a constant mass; the propellant is consumed at a rate governed by the equation

$$\dot{m} = -\frac{T_c}{I_{sp}\, g_0}, \tag{1}$$

This means that the inertia tensor $\mathbf{I}(t)$ is not constant throughout the descent. As the mass changes, the moment arms over which the thrust and aerodynamic forces act will change correspondingly. In addition, the propellant slosh will introduce an additional degree of freedom in the disturbance torques. A controller based upon a linear model of the system will accumulate increasing prediction errors as the time-to-burn increases, ultimately becoming unable to control the vehicle.

2.2. Challenge 2 — Sparse Terminal Rewards

The most informative signal accessible to the landing controller – the success of the landing – does not become available until the very last instant of a trajectory which may take hundreds of seconds. During early training, most episodes end in failure almost every time, providing no gradient information about which decisions were helpful and which were unhelpful. The standard DRL approach can stall for millions of episodes in this situation, accomplishing virtually nothing.

2.3. Challenge 3 — Distribution Shift and Sim-to-Real Transfer

When a policy is trained in simulation, it learns to optimize over the specific statistical regularities of the simulator. Deploying the policy in the real world breaks these regularities. The actuators may not respond as the model predicts. The noise in the sensors may not be Gaussian. The coefficients of the air may not be what the model predicts. The real air may not be the standard model. The policy makes decisions based on the correlation between the observations and the outcome. The policy will work as long as the correlation holds. But we cannot be sure of this.

2.4. Challenge 4 — No Formal Safety Guarantees

A policy with high average landing success rate does not offer any guarantees for any particular trajectory. Aerospace certification needs hard guarantees, such as the gimbal will never swing through its structural limit, and the landing rate will never exceed what the landing structure can handle. Statistics from a variety of training rollouts do not meet either of these needs. Certification schemes need guarantees, and DRL does not provide them.

2.5. Challenge 5 — Closed-Loop Stability is Unverified

A policy can meet all discrete safety constraints and yet, when combined with the dynamics of the system. This can result in instability, oscillation, or a limit cycle. The extension of classical Lyapunov stability theory to a nonlinear system with a black-box neural network as its feedback is not trivial, and until recently, it was practically impossible.

2.6. Challenge 6 — Black-Box Opacity and Explainability

When the neural network controller sends an anomalous command, it is difficult to determine what observations led to the decision or whether the causal reasoning is related to any physically meaningful phenomenon. This is problematic for two reasons: it impedes the anomaly diagnosis necessary for safety certification, and it makes engineering improvement less efficient.

2.7. Challenge 7 — Single-Point Hardware Failure Fragility

A monolithic architecture where all sensor information is broadcast to all actuator outputs through a single network lacks the ability to handle failures gracefully. The failure of a single actuator can cause an abrupt failure of the entire system because the policy does not have any internal model of which actuators are functioning or how authority can be distributed among the functioning ones.

3. Mathematical Foundations

The following sections build on the CVL architecture's mathematical core.

3.1. Six-Degree-of-Freedom Dynamics

The complete mechanical state of the descending vehicle is specified by its position $\boldsymbol{r} \in \mathbb{R}^3$, its velocity $\boldsymbol{v} \in \mathbb{R}^3$ in the inertial frame, its orientation via an attitude quaternion $\boldsymbol{q} \in \mathbb{R}^4$, its angular rate $\boldsymbol{\omega} \in \mathbb{R}^3$ about its own axes, and its instantaneous wet mass $m \in \mathbb{R}^+$. The North-East-Down frame translation dynamics are described by

$$m(t)\frac{d\boldsymbol{v}}{dt} = \boldsymbol{T}(t) + \boldsymbol{F}_{\text{aero}}(\boldsymbol{v}, h, \alpha) + \boldsymbol{F}_{\text{gravity}}(\boldsymbol{r}) + \boldsymbol{F}_{\text{wind}}(h, t). \tag{2}$$

The rotation dynamics are described by Euler's equations, extended to account for the variable inertia tensor to capture the effect of the mass shift discussed in Section 2. Thus,

$$\mathbf{I}(t)\frac{d\boldsymbol{\omega}}{dt} = M_{\text{thrust}}(\delta_p, \delta_y, T_c) + M_{\text{aero}}(\alpha, \beta, \boldsymbol{\omega}) + M_{\text{slosh}}(t) - \boldsymbol{\omega} \times \big(\mathbf{I}(t) \cdot \boldsymbol{\omega}\big). \tag{3}$$

The orientation dynamics are described by quaternions,

$$\dot{\boldsymbol{q}} = \tfrac{1}{2}\,\Xi(\boldsymbol{q})\,\boldsymbol{\omega}, \tag{4}$$

as opposed to Euler angles, to circumvent the notorious Euler angles' singularities, as discussed in [Shuster, 1993].

Remark 1. *A useful initial programming exercise is to implement Equations* (2)–(4) *in Python, using* `scipy.integrate.solve_ivp`*, with a fixed inertia tensor and without any aerodynamic forces. This reduced model captures the essence of the guidance problem without any of the complexity, making it an excellent starting point.*

3.2. Markov Decision Process Formulation

The guidance task is formulated as an episodic Markov Decision Process $\mathcal{M} = (S, A, P, R, \gamma)$. Here, S is a continuous state space encoding vehicle kinematics and estimated environmental disturbances; $A = [\delta_p, \delta_y, T_c]$ is the continuous action space (pitch gimbal, yaw gimbal, and throttle command); $P(s' \mid s, a)$ is the transition distribution induced by the vehicle dynamics; $R(s, a)$ is the reward signal; and $\gamma \in [0, 1)$ is the discount factor. The policy $\pi_\theta : S \to A$ is a neural network with parameters θ, and is trained to maximise the expected sum of discounted rewards:

$$J(\theta) = \mathbb{E}_\pi \left[\sum_t \gamma^t R(s_t, a_t) \right]. \tag{5}$$

Among the available policy-gradient algorithms, Proximal Policy Optimisation (PPO; Schulman et al. 2017) is a suitable starting point for students in this domain. Its on-policy update rule is conceptually transparent, its behaviour is predictable across a wide range of hyperparameter settings, and high-quality open-source implementations (e.g. Stable-Baselines3) are readily available.

3.3. Reward Function Design

The CVL framework uses a composite reward that weighs five qualitatively distinct objectives:

$$R(s_t, a_t) = \underbrace{w_1 R_{\text{terminal}}\, \mathbf{1}_{[t=T]}}_{\text{sparse accuracy}} + \underbrace{w_2 R_{\text{load}}(a_t)}_{\text{structural safety}} + \underbrace{w_3 R_{\text{fuel}}(t)}_{\text{efficiency}} + \underbrace{w_4 R_{\text{attitude}}(s_t)}_{\text{stability}} + \underbrace{R_{\text{shaping}}(s_t, s_{t+1})}_{\text{dense guidance}}. \tag{6}$$

Dense intermediate feedback is supplied through a potential-based shaping term $R_{\text{shaping}} = \gamma\Phi(s_{t+1}) - \Phi(s_t)$, where

$$\Phi(s) = -\lambda \|\boldsymbol{r} - \boldsymbol{r}_{\text{pad}}\| - \mu \|\boldsymbol{v}\|. \tag{7}$$

The key property of this form, established by Ng et al. [1999], is that potential-based shaping cannot change the identity of the optimal policy — the agent is guided more informatively during training without being pushed toward any policy that differs from the one optimising the original objective. Every student working on reward design should internalise this result before modifying reward functions. The terminal component takes the form

$$R_{\text{terminal}} = \exp\left(\frac{-\|\boldsymbol{r}_T - \boldsymbol{r}_{\text{pad}}\|^2}{\sigma_r^2} \right) \cdot \exp\left(\frac{-\|\boldsymbol{v}_T\|^2}{\sigma_v^2} \right) \cdot \exp\left(\frac{-\theta_T^2}{\sigma_\theta^2} \right), \tag{8}$$

and the structural load penalty penalises both gimbal deflection magnitude and rapid throttle changes:

$$R_{\text{load}} = -\kappa_\delta \left(\delta_p^2 + \delta_y^2 \right) - \kappa_{\dot{T}} \left(\frac{dT_c}{dt} \right)^2. \tag{9}$$

3.4. Structural Causal Models and the SC-MDP

A Structural Causal Model (SCM) $\mathcal{C} = (\boldsymbol{V}, E, F, P_U)$ enriches an ordinary probabilistic model by attaching a directed acyclic graph G that explicitly encodes the direction of

causal influence among the variables in the system. The operationally critical concept is the *do-operator*: the quantity $P(Y \mid \mathrm{do}(X = x))$ is the distribution of Y that would result from forcibly setting X to x — an intervention that severs all incoming edges to X in G. This interventional probability is not the same as the observational conditional $P(Y \mid X = x)$ wherever unobserved confounders are present [Pearl, 2009].

Remark 2. *The Atmospheric density $\rho(h)$ is a good example of why this is important. In a training simulation, ρ is related to altitude, so a standard DRL policy may use ρ as a substitute for altitude when calculating the value function. That correlation ceases to exist as soon as the atmospheric profile changes during deployment. CVL makes the policy learn the real causal relationship between aerodynamic forces and flight state by encoding ρ as a confounder in the causal DAG and using the backdoor criterion to block its false gradient contribution. This way, the policy does not rely on a proxy that might not work in a different domain.*

3.5. GR(1) Temporal Logic Safety Specification

Linear Temporal Logic (LTL) is a logic for expressing formulas about the properties that must be satisfied over time. We focus on two LTL formulas: $G\phi$ which denotes "ϕ holds at every future time step" and $GF\phi$ which denotes "ϕ holds infinitely often in the future." The GR(1) fragment restricts the formulas to the following form:

$$\varphi = \left(\bigwedge_i G(\psi_i) \wedge \bigwedge_i GF(a_i) \right) \rightarrow \left(\bigwedge_j G(\rho_j) \wedge \bigwedge_j GF(\pi_j) \right), \tag{10}$$

This can be interpreted as follows: if the environment satisfies its behavioral assumptions (the antecedent), then the system must satisfy the safety invariants ρ_j and the liveness goals π_j (the consequent). The six invariants ρ_j in the rocket landing domain express the system requirements on the limits of gimbal motion, the minimum propellant margin, the monotonic decrease in altitude, the maximum descent rate, the sequencing of landing gear deployment, and the shutdown of the engines. The GR(1) fragment has the advantage of polynomial-time synthesis via Binary Decision Diagrams, thus keeping the verification cost commensurate with the constraints of deployment in the system.

3.6. Neural Lyapunov Stability Certification

A function $V : S \rightarrow \mathbb{R}^+$ is said to be a Lyapunov certificate for the equilibrium point $\boldsymbol{s}^*$ if the following three conditions hold on the domain Ω:

$$V(\boldsymbol{s}^*) = 0, \quad V(\boldsymbol{s}) > 0 \; \forall \, \boldsymbol{s} \in \Omega \setminus \{\boldsymbol{s}^*\}, \quad \dot{V}(\boldsymbol{s}) = \nabla V(\boldsymbol{s})^\top f(\boldsymbol{s}, \pi(\boldsymbol{s})) \leq -\alpha V(\boldsymbol{s}). \tag{11}$$

Note that the third condition is the key one. This condition ensures that the value function V decreases along every closed-loop trajectory at a rate proportional to the current value. This ensures asymptotic convergence to the equilibrium point $\boldsymbol{s}^*$. Chang et al. [2019] showed that the neural network can be learned simultaneously with the policy to satisfy all three conditions. They employed an adversarial SMT solver to obtain counterexample states where the conditions were violated.

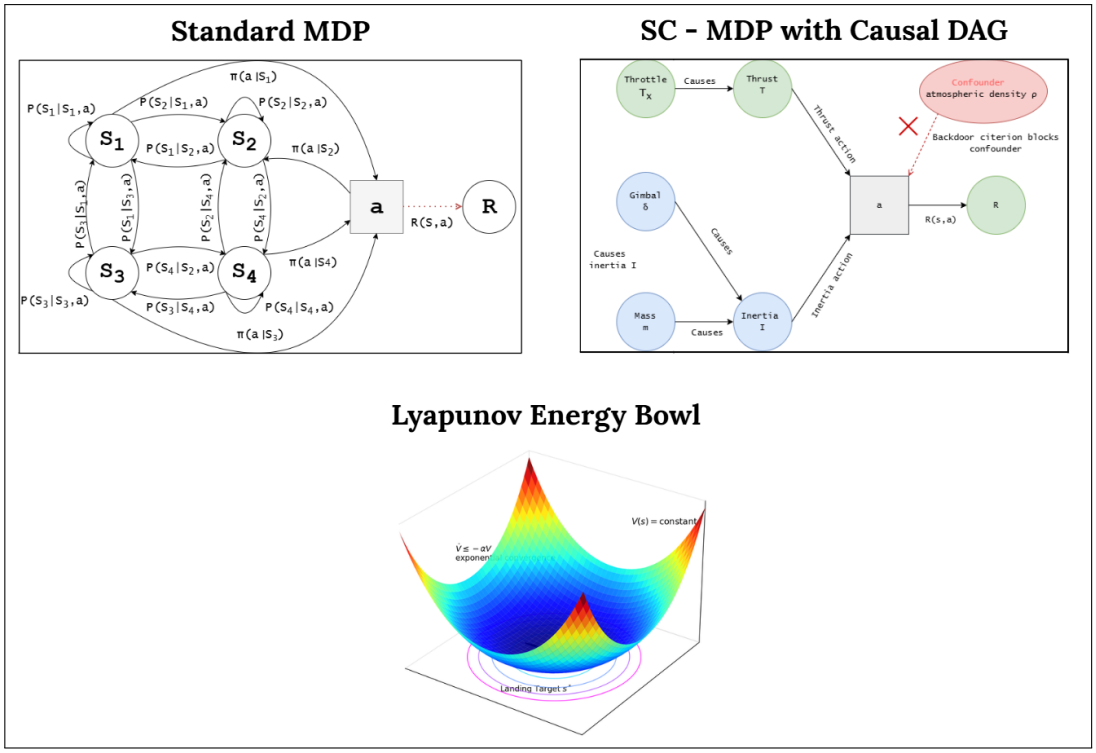

Figure 1: (a) A standard MDP with transition probabilities. (b) SC-MDP with a causal DAG overlay that shows how to find and block confounders. (c) The Lyapunov energy bowl shows how the landscape gets lower as it gets closer to the landing target state.

4. The CVL Architectural Blueprint

Now that we have set up the problem structure and the math tools, we can move on to the CVL framework itself. Instead of telling the story of the architecture from the top down and then explaining why choices were made, each layer is described along with the specific challenge from Section 2 that led to its inclusion. This makes it impossible to separate design reasoning from design description.

4.1. Overview: Eleven Layers, Three Tiers

The CVL architecture comprises eleven distinct layers, which are divided into three tiers. The **Perception Tier** (L1–L2) receives sensor data as input and returns a compact representation thereof along with quantified uncertainty. The **Cognition Tier** (L3–L5) then processes this estimate via causal filtering, world modeling, and hierarchical policy execution. Finally, the **Control and Safety Tier** (L6–L11) ensures hard safety constraints are satisfied while providing stability guarantees and online adaptability, as well as a human-readable causal trace for all control actions. Each layer is given a distinct responsibility as a design principle in itself. This ensures auditability and facilitates localisation of failure.

4.2. Perception and Sensor Fusion (L1–L2)

The CVL system integrates four different sensor modalities: an inertial measurement unit (IMU), a radar altimeter, a LiDAR terrain mapper, and a nadir-facing camera. These modalities are then fused together via a Transformer-based architecture. This involves first encoding each modality within a separate encoder before then employing the cross-modal attention mechanism to determine the weightage to be given to each modality in a particular context. This will automatically decrease weightage for a LiDAR scan affected by landing plume dust or increase weightage for the altimeter during a GPS-denied final approach.

In addition to the learned approach, an 18-state error-state Kalman filter (ESKF) is used as a classical navigation solution and both as a fallback and as input feature for the learned encoder. The overall input processing involves four steps:

1. *Sensor fusion via ESKF* at 200 Hz, producing estimates of position, velocity, and attitude together with a covariance matrix that quantifies current navigation uncertainty.
2. *Modality-specific encoding*: the IMU window (last 250 ms at 200 Hz) passes through a Temporal Convolutional Network (TCN); the LiDAR point cloud through PointNet++; and the camera frame through MobileNetV3-Small.
3. *Cross-modal Transformer fusion* (3 layers, 4 attention heads, $d_{\text{model}} = 128$), which yields a fused embedding $\boldsymbol{z}_t \in \mathbb{R}^{128}$.
4. *Causal GNN encoding* via four rounds of directed message passing on the causal DAG, which produces the causal state vector $\hat{\boldsymbol{s}}_t \in \mathbb{R}^{64}$ consumed by the downstream policy.

4.3. Causal State Estimation (L3)

The fused embedding from L2 is processed by a Graph Neural Network (GNN) operating directly on the causal DAG G derived from first-principles vehicle dynamics. The GNN performs message passing strictly in the direction of causal edges — influence propagates from causes to effects, never in reverse. The per-node attention weights produced in this process serve a dual role: they form the causal state representation that feeds the policy, and they provide the raw attribution data used downstream in the explainability layer L10. The starting point is that incorporating established causal structure as an inductive bias into the learned representation significantly diminishes sample complexity and enhances robustness in the face of distribution shifts, in contrast to an unconstrained encoder [Schölkopf et al., 2021].

4.4. Bayesian World Model with Physics-Informed Neural Networks (L4)

The problem of the Sim-to-Real gap is also addressed in the L4 layer by using two different approaches. First, we reduce the problem by using a group of five neural networks to estimate the residuals in the corrections to the nominal physics model, as described in the physics-informed paper by Raissi et al. [2019]. Since the nominal model already captures the dominant system dynamics, the networks only need to learn the residual errors, making the learning task significantly more efficient and data-efficient. Secondly, the group also estimates the parameters in the physics equations, such as the effective

specific impulse, the density scaling factor, the drag offset, and the gimbal lag. These parameters are estimated using a Sigma Point Kalman Filter. When the system encounters flight conditions outside the training distribution, the parameter estimator will identify the change and correct the world model's belief distribution. An important concept in the context of the paper and the world model is the fact that when the ensemble has a high disagreement in the predictions, the disagreement represents epistemic uncertainty, indicating places where the model lacks.

4.5. Hierarchical Policy and RAJS Curriculum (L5)

Instead of training a single policy to control the entire descent process, the policy in CVL is hierarchical in structure. A deterministic finite automaton (DFA) controls the altitude and velocity with respect to a predefined threshold and selects one out of six phase-specific sub-policies accordingly. This decomposition simplifies the curriculum learning process for the landing phase.

The Random Annealing Jump Start (RAJS) curriculum addresses the cold start problem of the terminal descent phase by defining a decaying teacher horizon as follows:

$$H_{\text{guide}}(k) = H_{\max} \cdot \exp\left(-\alpha \, \frac{k - k_{\text{start}}}{k_{\text{anneal}}}\right). \tag{12}$$

In the initial stages of training, a classical LCvx guidance solution [Acikmese and Ploen, 2007, Blackmore et al., 2010] is used to guide the vehicle almost to the pad, and the RL agent will only control the vehicle in the final seconds. However, as training progresses, the handover altitude increases until the RL agent controls the entire descent profile. A key aspect of the RAJS curriculum is that the RL agent is not trained to mimic the teacher's policy. Instead, it starts off with a nearly viable state provided by the teacher's policy. This avoids the distributional mismatch caused by behavioral cloning while providing a significant speed-up in the efficiency of exploration.

4.6. Multi-Agent Cooperative Actuator Control (L6)

Instead of using a monolithic output network, CVL assigns one cooperative agent per actuator system: the main engine, the reaction control system, the grid fins, and the landing legs. These agents share the latent communication vector, which enables implicit information sharing between the agents without the need for any hard-coded failure protocol. When one of the actuator systems fails, the failure will propagate through the communication space, with the other agents adapting their output distribution in response without the need for the redistribution process to be explicitly coded. The credit assignment between the agents is facilitated using the counterfactual advantage estimates provided by the COMA algorithm [Lowe et al., 2017], which enables the individual contribution of each agent to the shared reward signal to be calculated.

4.7. Formal Verification Engine (L7)

A three-stage pipeline has been implemented to ensure runtime safety. First, the continuous state space is discretized to a finite cell partition by zonotope over-approximation, and the output of the neural network is bounded over each cell using interval arithmetic. This abstract transition system is then composed with the six GR(1) safety invariants

as previously defined in Section 3.5. Finally, a polynomial-time symbolic synthesis is employed to determine if a strategy exists to satisfy all the invariants. At runtime, every action is checked against the synthesized strategy by a lightweight monitor (18 μs per call) at 200 Hz. If an unsafe action is projected, it is projected onto the nearest safe alternative before it is sent to the actuator command interface.

4.8. Neural Lyapunov Certifier (L8)

The Neural Lyapunov Certifier (L8) trains an Input Convex Neural Network (ICNN) to provide a certificate of exponential convergence to the landing target. ICNNs have an advantageous architecture that guarantees positive definite outputs. By design, all weights on the input pathway to the hidden layers are non-negative. This guarantees the convexity of the function over the entire input space and thus a unique minimum at the equilibrium point to which we wish to converge [Chang et al., 2019]. Once again, we employ the co-training procedure defined in Section 3.6.

4.9. Online Meta-Adapter (L9)

On top of the MAML framework developed in Finn et al. [2017], a light-weight outer MLP is updated in flight at 2 Hz based on observations of recent trajectory history. However, two constraints are placed on this adaptation to prevent degradation of the safety properties proved in L7 and L8: a hard constraint on the norms of weight updates for the adapter is enforced to prevent catastrophic forgetting of the base policy, while a complete shutdown of adaptation is enforced when any GR(1) safety invariant approaches its threshold. This creates a two-tiered control architecture in which safety is guaranteed by a safety-invariant core while performance is maximized by continuous adaptation within a certified envelope.

4.10. Causal Attribution Interface (L10)

The Causal Attribution Interface is run at 10 Hz and calculates a score for all nodes in the causal graph for every control action based on their relative contribution to the control action taken. This is done using the do-calculus framework to ensure that spurious correlations are not present. This per-decision attribution of control actions answers the question "which physical variable most caused this command?" providing a physically meaningful explanation rather than a purely statistical one. This capability is important for engineering alalysis and directly supports interpretability requirements for AI Concepts of Operations (ConOps) as defined within the EASA AI certification standards [EASA, 2023].

5. The RL Training Pipeline

The CVL training process follows a loop having four stages, repeated millions of times until the policy converges to a correct and reliable landing behaviour. These stages consist of simulation rollout to generate experience, state preprocessing and encoding, reward computation, and policy improvement via PPO.

5.1. Stage 1 — Data Collection via Simulation Rollout

The training process starts by randomly sampling the initial state. This consists of the altitude, which is uniformly sampled within the range 5,500 m to 8,000 m, the lateral offset, which is up to 2,500 m away from the pad, and the vertical velocity, which ranges from -250 m/s to -420 m/s. Then, the 6-DOF dynamics proceed in a sequence by using a fourth-order Runge-Kutta integrator with a time step size of 5 ms. Additionally, wind disturbances are sampled using the Dryden turbulence model, and Gaussian noise is added to the sensor signals before the observation state is generated. By running 1,024 parallel environment instances in parallel, the process generates approximately 200,000 transitions per real-world second. Besides the randomization of the initial state, the training process also randomizes the physical parameters consisting of the atmospheric density, the engine's specific impulse, the gimbal time constant, and the drag coefficient. These are independently randomized at the start of every rollout. This process is the main method by which the policy learned will be robust to the unmodelled dynamics it will encounter on the real-world system.

5.2. Stage 2 — State Preprocessing and Encoding

The preprocessing pipeline described in Section 4.2 builds structure into the raw observation at each stage: the ESKF injects temporal consistency, the modality-specific encoders inject domain-appropriate representations, the Transformer injects cross-modal relational context, and the causal GNN injects the physical causal structure of the vehicle dynamics. By the time the state vector reaches the policy head, it encodes a rich, structured, physically grounded summary of the vehicle's situation rather than a concatenation of raw sensor readings.

5.3. Stage 3 — Reward Computation

At each timestep the scalar reward r_t is formed from the five terms of Equation (6). Practitioners training their first landing agent frequently encounter two characteristic failure modes in reward weight tuning: giving w_1 too large a value relative to the dense terms effectively reinstates the sparse reward problem, while over-weighting the shaping potential causes the agent to prioritise smooth trajectories at the expense of actually reaching the target pad. Both failure modes are instructive and worth encountering deliberately.

5.4. Stage 4 — PPO Policy Update

After 4,096 transitions have been collected, the accumulated experience is used to update the policy network parameters θ.

5.4.1. Advantage Estimation

Advantage estimates are computed using Generalised Advantage Estimation (GAE-λ):

$$\hat{A}_t = \sum_{\tau=t}^{T} (\gamma\lambda)^{\tau-t}\delta_\tau, \qquad \delta_\tau = r_\tau + \gamma V(s_{\tau+1}) - V(s_\tau). \tag{13}$$

In the causal extension of CVL, the advantage estimate is corrected by subtracting the component attributable to confounders in the causal DAG, so that gradient updates credit

only causally attributable differences in performance rather than spurious correlations that happen to be present in the collected batch.

5.4.2. Clipped Surrogate Objective

The policy is improved by maximising the PPO clipped objective:

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\Big)\Big], \tag{14}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\mathrm{old}}}(a_t|s_t)$. The clipping mechanism prevents the policy from moving too far from its previous iterate in any single update, a stability property that is particularly valuable when training on physical systems where large policy changes can destroy hard-won behavioural structure.

5.4.3. Mini-batch Gradient Updates

The 4,096-transition buffer is broken into 512-transition mini-batches. The Adam optimizer works on each mini-batch one at a time, and the whole buffer is used again for 10 gradient epochs before new data are gathered.

5.4.4. RAJS Teacher Handoff

The RAJS scheduler uses Equation (12) to figure out $H_{\mathrm{guide}}(k)$ at the beginning of each episode. The LCvx guidance solution controls the vehicle for the first H_{guide} time-steps. After that, the PPO policy takes over until the vehicle lands. Training goes on until both the success rate and the CEP_{50} landing accuracy level off. This usually takes about 500 million environment steps in the full CVL configuration.

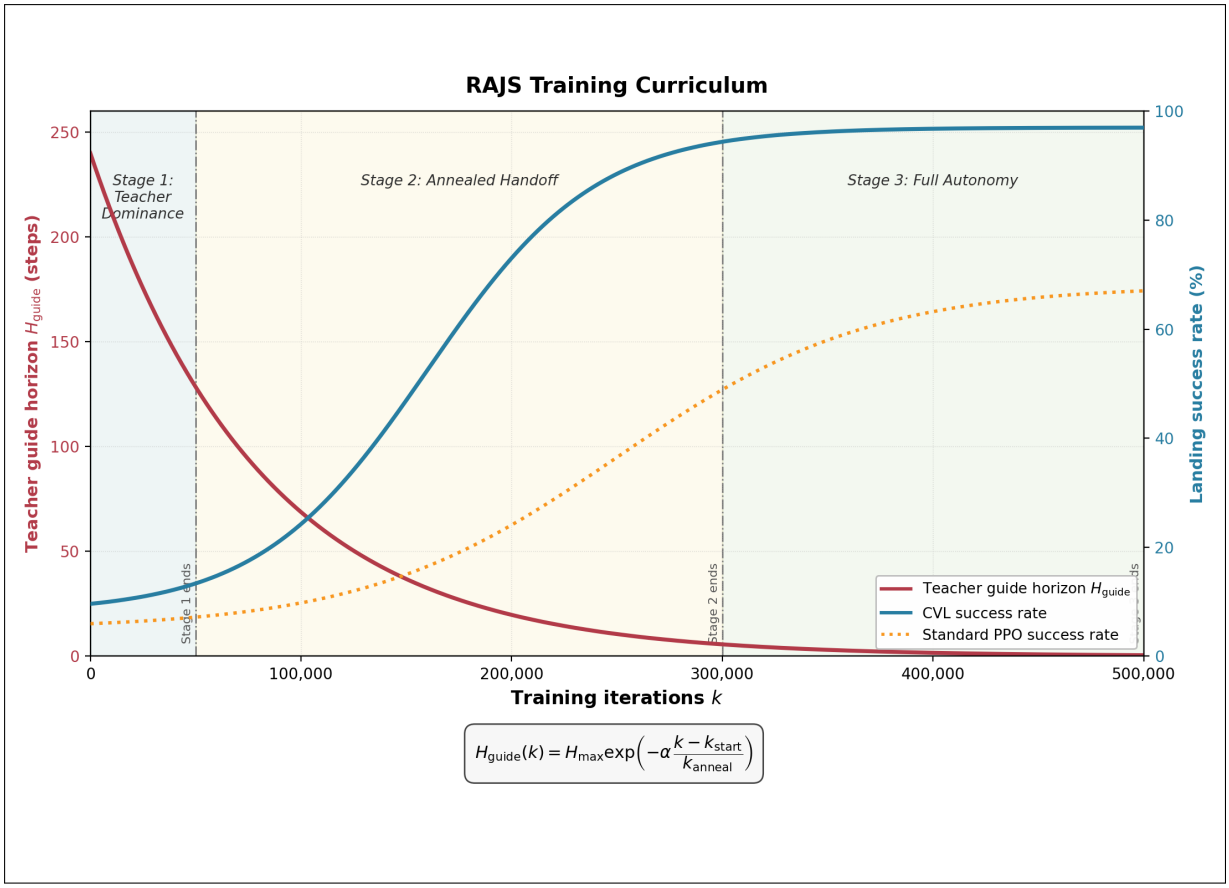

Figure 2: The RAJS training curriculum has three stages that show how the teacher guide horizon $H_{\text{guide}}(k)$ gets smaller and smaller. As this happens, the learned PPO policy takes over control from the classical LCvx teacher. The width of domain randomization goes up at the same time. [**Note: The curves are just examples; the exact values are not based on experiments.**]

6. A Progressive Implementation Roadmap for Student Teams

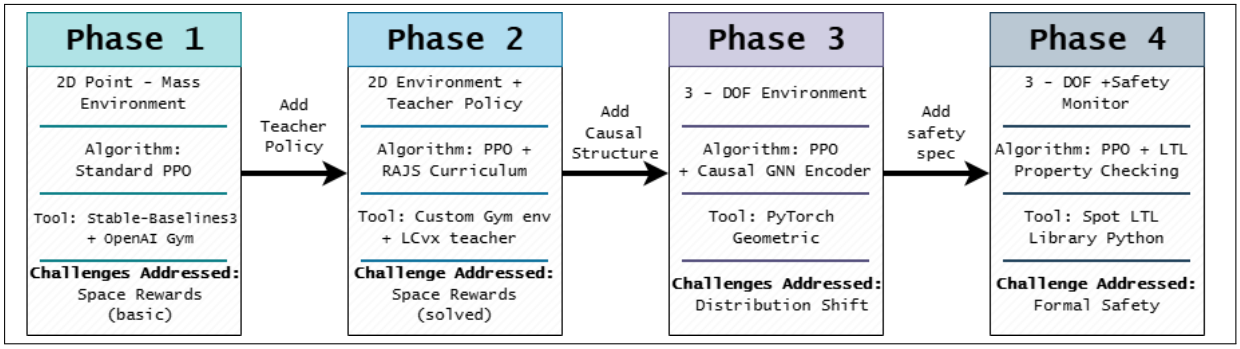

Figure 3: A four-phase plan for students to follow. Each phase adds one new idea while building on what has already been done. It goes from a 2-D point-mass environment to a 3-DOF system with causal GNN state estimation and LTL safety verification.

6.1. Phase 1 — 2-D Point-Mass Landing

Goal: Develop the basic formulation while still posing the agent with the fundamental RL challenge.

Implement a 2-D vertical and lateral point mass landing problem in Python, either from scratch using OpenAI Gym or as a proxy for a modified version of `LunarLander-v2`. The state vector will be $[x, y, v_x, v_y, m]$. The agent will control thrust force and direction. Implement a standard PPO agent using Stable-Baselines3 and let the sparse reward problem emerge spontaneously. Observe how the agent fails to make progress for tens of millions of steps. This demonstrates why dense rewards are required. No amount of theory can replace experiencing this.

Recommended reading: Sutton and Barto [2018], Ch. 1–4 (theoretical foundations of MDPs); Schulman et al. [2017] (the PPO algorithm); Ng et al. [1999] (reward shaping theory).

6.2. Phase 2 — Reward Shaping and RAJS

Goal: Show the power of curriculum learning in dramatically speeding up learning.

Implement a classical proportional derivative guidance law as our RAJS teacher. Apply the annealing schedule as specified in Equation (12). Compare learning curves (e.g., landing success rate vs. steps taken) for a vanilla PPO run and a PPO run with our RAJS curriculum. The acceleration will be immediately visible on the training curves, providing a concrete motivation for the structured curricula.

Recommended reading: Acikmese and Ploen [2007] for the physics underlying our teacher policy; the behavioural cloning vs. RL literature for understanding why our RAJS approach avoids distributional issues in imitation learning.

6.3. Phase 3 — 3-DOF with Causal Structure

Goal: Show that the causal state encoding leads to better out-of-distribution robustness.

Add a minimal causal DAG with five nodes to the extended environment with 3-DOF and train two agents in parallel with the message passing GNN. One will be trained with the original observation vector, and the other will be trained with the causal state vector. Test both agents with a distribution shift by increasing the density by 20%. This will show the value added by the causal inductive bias.

Recommended reading: Schölkopf et al. [2021] (causal representation learning); Pearl [2009] (do-calculus and backdoor criterion); PyTorch Geometric documentation.

6.4. Phase 4 — Formal Safety Specification

Goal: Write the safety specifications in a verifiable formal language and validate them on a trained policy.

Define two LTL safety properties for the 3-DOF environment. First, the thrust must be non-zero whenever the altitude is above 10 m. Second, the touchdown velocity must be within 2 m/s. Using the Spot library (Python bindings), validate the satisfaction of the properties on the sampled trajectories generated by the trained policy. When the

properties are violated, which they will be, add a penalty to the reward and retrain until the properties are validated on a separate evaluation set.

Recommended reading: Baier & Katoen, *Principles of Model Checking*, Chapters 1–3; Spot library documentation; Kress-Gazit et al. [2009] for the connection to reactive synthesis.

7. Connections to Existing Literature and Open Problems

7.1. What Industry Has Done

Commercial RLV operators have demonstrated autonomous landing at scales and repeatability rates that validate the engineering premise of this paper. Patent filings, conference presentations, and independent analysis of observed vehicle behaviour collectively point to heavy reliance on model predictive control, convex trajectory optimisation, and classical GNC architectures augmented with machine learning for specific subtasks such as terrain-relative navigation and wind field estimation. To the authors' knowledge, no open publication describes a fully integrated, formally verified neural landing controller with causal interpretability — which is precisely the synthesis that CVL proposes as a conceptual target.

7.2. Open Research Problems

Scalable formal abstraction. The zonotope-based approach to state-space abstraction scales polynomially in the number of abstracted variables. Finding abstraction methods that yield tighter bounds with better asymptotic behaviour remains an active and practically important research problem.

Causal discovery from flight data. CVL assumes the causal DAG is supplied by domain knowledge derived from physics. Automatically recovering causal structure from observational flight data — using methods such as PC, FCI, or NOTEARS — is an open problem with direct relevance to any deployment that lacks a fully validated physics model.

Certifiable meta-learning. The online meta-adapter in L9 is currently constrained by heuristic norm bounds. Deriving theoretically grounded update constraints that provably preserve the formal verification certificates established in L7 and L8 is an important unsolved problem at the intersection of meta-learning and formal methods.

Multi-agent formal verification. CVL's GR(1) synthesis is applied to individual agent policies independently. Extending formal verification to cooperative multi-agent systems with shared communication channels is an emerging research frontier that has direct applicability to the actuator-agent architecture described in Section 4.6.

8. Discussion

8.1. Pedagogical Design Principles Applied

Four specific pedagogical design principles guide the organisation of this paper. **Problem-first reasoning:** every element of CVL is introduced in the context of the particular

failure mode which makes it necessary; architectural decisions are never presented in isolation. **Progressive complexity:** the four-phase implementation roadmap introduces exactly one level of abstraction at a time; in every phase, we get a working artifact which the next phase builds upon. **Mathematical precision without unnecessary abstraction:** the full formal notation of the respective domain is employed, but only after a natural language description of what the mathematics describes and why it is the correct mathematics for the task at hand. **Honest scoping:** the paper makes clear what is synthesis, what is literature review, and what remains to be done. The former is not a weakening but actually strengthens the paper as a pedagogical document.

8.2. Limitations of This Work

CVL is aimed to be a theoretical design synthesis rather than a system that is experimentally validated. The architecture draws on components that have each been validated independently in the literature, but the integrated eleven-layer system has not been implemented or tested. Several specific design choices — the GNN depth and width, the norm bound applied to the meta-adapter, and the cell resolution of the zonotope abstraction — are offered as well-motivated starting points rather than optimised configurations. The six-node causal DAG used in the worked examples is a deliberate simplification; a production implementation would require careful causal discovery work to validate the DAG structure against actual flight telemetry. Finally, the formal verification coverage figure cited elsewhere in the literature for similar zonotope abstraction approaches should not be read as a validated claim for the specific CVL design presented here.

9. Conclusion

This paper has offered a structured route into one of the most technically rich problems in modern autonomous systems engineering. Starting from a catalogue of the seven ways that naïve DRL-based landing controllers fail, we developed the mathematical tools needed to address each failure mode and assembled those tools into the eleven-layer CVL architectural blueprint. A four-phase implementation roadmap then translates this blueprint into a concrete series of open-source projects that a student team can execute.

The important point the paper makes is that it is not possible to build safety-critical autonomous systems by combining machine learning techniques that are fashionable at a given time. Sound engineering practice involves understanding what each component guarantees, what assumptions those guarantees depend upon, and how those guarantees combine when combining components. CVL is proposed as a reasoning system to build this understanding, not to build the understanding itself.

Students completing the four phases will have hands-on experience with DRL, causal GNNs, reward shaping, and LTL property checking. This is a skillset that is genuinely rare and becoming increasingly valuable to the development of certified autonomous aerospace systems. The open problems listed in Section 7.2 outline possible research directions that student projects can take to make important contributions to the field.

References

Acikmese, B. and Ploen, S.R. (2007). Convex programming approach to powered descent guidance. *AIAA Journal of Guidance, Control, and Dynamics*, 30(5):1353–1366.

Blackmore, L., Acikmese, B., and Scharf, D.P. (2010). Minimum-landing-error powered-descent guidance for Mars landing using convex optimization. *AIAA Journal of Guidance, Control, and Dynamics*, 33(4):1161–1171.

Chang, Y.-C., Roohi, N., and Gao, S. (2019). Neural Lyapunov control. *Advances in Neural Information Processing Systems*, 32.

European Union Aviation Safety Agency (2023). *EASA Artificial Intelligence Roadmap 2.0*. Technical Report, EASA.

Finn, C., Abbeel, P., and Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. *Proceedings of the International Conference on Machine Learning (ICML)*, 70:1126–1135.

Gaudet, B., Linares, R., and Furfaro, R. (2020). Deep reinforcement learning for six degree-of-freedom planetary landing. *Advances in Space Research*, 65(7):1723–1741.

Kress-Gazit, H., Fainekos, G.E., and Pappas, G.J. (2009). Temporal-logic-based reactive mission and motion planning. *IEEE Transactions on Robotics*, 25(6):1370–1381.

Lowe, R. et al. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in Neural Information Processing Systems*, 30.

Malyuta, D. et al. (2022). Advances in trajectory optimization for space vehicle control. *Annual Reviews in Control*, 52:282–315.

McMahan, H.B. et al. (2017). Communication-efficient learning of deep networks from decentralized data. *Proceedings of AISTATS*, 54:1273–1282.

Ng, A.Y., Harada, D., and Russell, S.J. (1999). Policy invariance under reward transformations: theory and application to reward shaping. *Proceedings of the International Conference on Machine Learning (ICML)*, 16:278–287.

Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*, 2nd edition. Cambridge University Press, Cambridge, UK.

Raissi, M., Perdikaris, P., and Karniadakis, G.E. (2019). Physics-informed neural networks. *Journal of Computational Physics*, 378:686–707.

Schölkopf, B. et al. (2021). Toward causal representation learning. *Proceedings of the IEEE*, 109(5):612–634.

Schulman, J. et al. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Shuster, M.D. (1993). A survey of attitude representations. *Journal of the Astronautical Sciences*, 41(4):439–517.

Sutton, R.S. and Barto, A.G. (2018). *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press, Cambridge, MA.